

Milestone 2

Database

1. Introduction

Kanban is a project management methodology that allows for organizing and prioritizing projects and their tasks using boards, lists, and cards. With the right Kanban board software, managers can focus on managing and prioritizing the team's tasks, keep the team in the loop, and control a steady workflow. This tool supports agile development and helps visualize and manage the workflow.

In this milestone, you are required to **improve** the backend of **your** Kanban software, including the business layer and the service layer. Given the requirements below, you will design these layers, implement them, and test them.

The current version of the kanban board will **add a database to your implementation.**

Please read the document thoroughly before starting the work. Also, you are required to think like professional software engineers and do the work in the right order: requirements elicitation (understanding in this case) ([Section 2](#)), design ([Section 3](#)), testing ([Section 4](#)), and implementation ([Section 5](#)). **We will check that you worked this way through your git repository** (elaborated below).

Note: milestones 1-3 are not related to assignment 0 in any way.

Goals

The goals of this milestone are:

- Practice the N-tier architecture.
- Practice teamwork & version control.
- Practice C#.
- Experience the right use of OOP with C#.
- Practice test-driven development.
- Understand and write a Low-Level Design (LLD).
- **Working with databases.**

2. Requirements

a. Business and Integrity Rules

1. A user is identified by an email and is authenticated by a password.
2. A valid password is in the length of 6 to 20 characters and must include at least one uppercase letter, one small character, and a number.
3. Each email address must be unique in the system and the [syntax of the email address](#) should be valid.
4. A task has the following attributes:
 - a. A unique id in the board (integer).
 - b. The creation time.
 - c. A due date.
 - d. A title (max. 50 characters, not empty).
 - e. A description (max. 300 characters, optional).
 - f. The assignee (the person who is assigned to the task).
5. A board has a unique id in the system (integer), a name, and three columns: 'backlog', 'in progress', and 'done'.
6. A user cannot have two boards with the same name (though the same board name can be used by two different users).

b. Functional Requirements

Users

7. The program will allow the registration of new users.
8. As a user, I want to be able to login using an email and a password, and logout.

Boards

9. As a user, I want to be able to create boards and own them.
10. By default, a user is created with no boards.
11. A board owner can delete a board (all of its tasks are deleted).
12. As a user, I want to be able to join and leave existing boards created by someone else.
There is no need to get permission from the board owner
13. A board owner can transfer the board ownership to another board member (a user who joined the board).
14. A board owner cannot leave the board.
15. Once a user leaves a board, all of her assigned tasks that are not done become unassigned.
16. Each column should support limiting the maximum number of its tasks.
17. By default, there will be no limit on the number of tasks.

18. The board will support adding new tasks to its backlog column only by any of the board members.
19. Tasks can be moved from 'backlog' to 'in progress' or from 'in progress' to 'done' columns. No other movements are allowed. Only the assignee can move tasks.

Tasks

20. A task that is not done can be changed by its assignee only.
21. All the task data can be changed, except for the creation time.
22. As a user, I want to be able to list my 'in progress' tasks (that I am assigned to) from all of my boards, so that I can plan my schedule.
23. An unassigned task can be assigned by any board member to any board member. By default, a task is unassigned. The assignment of an assigned task can be changed only by its assignee.

c. Non-functional Requirements

24. Service: Service layer functions should return a JSON. The arguments may accept either primitives or a JSON string. For example,

```
// returned string is in the JSON format
```

```
string foo1(int field1, string field2)
```

or

```
// returned string and argument are in the JSON format
```

```
string foo2(string json)
```

25. Logging:

The system must maintain a log that will track important events (e.g., a user created or a task was created) and all errors in the system. Note that "error" does not necessarily mean an exception that is "thrown" or "raised" by your runtime environment, but any situation which counts as invalid in our domain (more details on errors in the following requirement). Some guidelines:

- a. Tag entries with their severity/priority
 - b. Provide enough information to understand what went wrong, and where
 - c. Avoid storing entire stack traces directly in the log (they are more verbose than useful)
 - d. [log4net](#) should be used for logging. The configuration file for this package is given to you inside the template repository.
26. Error Handling is the process of responding to the occurrence, during computation, of errors – anomalous or exceptional conditions requiring special processing – often changing the normal flow of program execution. The program is expected to operate even when an error occurs, i.e.:
 - a. Handle any malformed input.

- b. Handle logic errors (e.g., log in for non-existing users, etc.).

27. Persistency:

- a. In this assignment, persistent data is stored in an SQLite database.
- b. The following data should be persisted: users and boards (including tasks/columns/etc.)
- c. Persisted data should be restored once the program starts.

3. Design instructions

Following the requirements, you must design your software and create a class diagram of the system. Here are some guidelines for this task:

1. Before starting your design (and of course writing your code), consider the following:
 - What are the operations that each entity in the system can/should do?
 - Where does each method belong? (login & logout/create & move task, etc.)
 - Can you think of new requirements that the client (the course staff) will **probably** request? Can you make the design and the code versatile enough to support such requests (of course without too much work on your side because we might not ask for it)?
2. N-Tier structure: pay attention to the right use of the N-tier model.
3. OOP principles: remember [OOP principles](#) and use them (Encapsulation, Abstraction, Interface, and Inheritance).
4. Having a persistent layer (and backup data in files) does not mean that the data objects (i.e., users and boards) are stored in files only. In fact, these data objects will probably have some logic and methods (e.g., the user object). Moreover, your data should be stored in the RAM for fast retrieval. The persistent layer should be called only upon system loading (for restoring the persistent data) and upon data updating (e.g., registration of a new user).

Use these guidelines to create an initial design of your application. Make sure that the names of the methods describe their purpose well.

During your work on the implementation, your design might (and probably will) need modifications. Whenever it happens — change the design and then change the code. Make sure that your code structure and class/methods/members names are aligned with your design.

4. Testing your software

a. How you will test your software

You will create a “BackendTests” project (elaborated in [Section 5.a](#)) that will be used for testing your implementation. This project will have one or more functions for testing each functional requirement. The “main” function of this project will call these functions one after the other.

The testing functions will use the service layer for communicating with the backend. They should cover all different aspects of the requirements (e.g., successful and unsuccessful login actions).

Each function must be preceded with an API docstring that says which requirement it tests. For example (see [Microsoft’s tutorial](#) for documenting your C# code):

```
/// <summary>
/// This function tests Requirement 7
/// </summary>
void testSuccessfulLogin() {
...
}
```

Note: these tests are called acceptance tests and they are “black-box” testing, since they are testing the functionality without knowing the implementation details. More on this topic in a few weeks.

b. How we will test your software

To automatically test your application, we included **an updated version of the “GradingService.cs”**.

- You **must** implement all its methods.
- You **must not** change the methods’ signature!
- You **may** add additional methods/fields/getter/setters if needed.

GradingService.cs will be used by us for testing your code, which is why you must not change the signatures of its methods. However, this file represents bad SE practices (low cohesion, unintuitive signatures, etc.). Therefore, your service layer should look nothing like this file. Your GradingService.cs file may either call the underlying layer directly or call other functions in your service layer.

5. Implementation Guidelines

a. Solution Structure

Your solution will include two projects:

1. A “library” project called “Backend”:
 - a. It will have **three** layers (folders) inside: ServiceLayer, BusinessLayer, and **DataAccessLayer**.
 - b. The project will not depend on other projects in the solution.
2. A “console” project called “BackendTests”:
 - a. It will have the backend project as a dependency.
 - b. It will use **your** service layer for communication (and not GradingService.cs).
 - c. Although the type of this project is “console”, it only needs to call the testing functions (no need to add console input/output from the user).

To be clear: you do not develop a presentation layer (e.g., GUI/console).

b. Files

1. Make sure you understand the [access modifiers](#) and use them correctly.
2. You may add folders, classes, and interfaces **as needed**.
3. Document thoroughly all **public** classes and public methods/getters. See for example the class “GradingService.cs”. To add such documentation, read [Microsoft’s tutorial](#) for documenting your C# code.
4. Pay attention to “[Magic numbers](#)”.
5. Classes, class members, and methods should not be static, except for logger and factory (when needed).

6. Your database filename must be “kanban.db” and it should be placed in the same folder of your compiled dll files (i.e., <project-directory>\bin\Debug\net6.0). The connection string must be relative (i.e.,

```
string path = Path.GetFullPath(Path.Combine(
    Directory.GetCurrentDirectory(), "kanban.db"));
);
```

As explained in Section 12, this database file should be copied also to the root directory of your project. The copied file should include the schema (tables) but must not include data.

c. Version Control

1. You are expected to work as a team and use a version control system (GitHub specifically). You should use the [GitHub Flow](#) workflow taught in class (branch per feature and using pull requests).
2. We will use the GitHub classroom. You must register and login to GitHub **using your BGU email**.
3. Join our GitHub classroom using this link: <https://classroom.github.com/a/Q1EYti>
4. You will continue using the repository of Milestone 1 in this milestone.
5. ~~One team member should create a team.~~
6. ~~The team name should be: 2021-2022:00, where the last two digits are the group number (from Moodle)~~
7. ~~The other team members should join the team.~~
8. ~~A repository will be created for the team automatically with the template repository supplied by us. You should use **only** this repo during the development of this milestone.~~
9. The root of your repository should contain:
 - a. A README .md file that its first line contains the IDs of the group members in the following format: ID1_ID2_ID3. The second line number should include the group number (from Moodle)
10. At the root of your repository, you must have a folder called "documents" that will contain your design document(s).
11. We will use your repository to check your workflow, your teamwork, how you balanced the work between all team members, etc. Therefore, do not work on a shared folder (e.g., Google Drive), learn how to use git correctly.

12. *Submission Dates, Deliverables, and Grading*

a. Deliverables

There are two submission deadlines for this assignment. At each deadline, you will submit your deliverables in two ways: (1) In Moodle, and (2) you will put the requested files in your repository. Both will be done before the deadline passes.

- **29/5 23:59** - Your design (class diagram), service layer, and tests.
- **23/6 23:59** - Your design docs and entire solution.

First submission - **29/5 23:59** - Your design (class diagram), service layer, and tests

The class diagram of your project will be added to the root of your master branch, by the name “documents/design.pdf” (can be created using, e.g., <https://draw.io>). **Your PDF must include a list of all changes, compared to the final pdf you submitted in milestone 1.** We will not grade your design, only check that you wrote it and put effort into doing it right.

In addition to the design, you will submit your service layer and the acceptance tests. **Note that for every method you add/change, with respect to the service layer of milestone 1, you must supply documentation and new/updated tests. As in milestone 1, there is no need to implement new functionalities for the first submission (though you may).**

We will not grade this part, but we will reduce points if the submitted files do not follow the instructions.

Second submission - 23/6 23:59 - Your design docs and entire solution

This submission will include your entire solution and your updated design. Specifically:

1. Your “documents/design.pdf” file will be updated with your latest class diagram. The file should also include a list of all the changes you made from the initial diagram, with short explanations of why you performed these changes.
2. For your Moodle submission, zip your entire repository including the documents directory but not including bin and obj directories. Upload the zip file to the Moodle assignment.
3. You should have a DB file named “kanban.db” at the root of your project. This file should be contained in the ZIP file at submission. Make sure the database has the tables (schema) and that the tables are empty (from rows) when submitting.

b. General Submission Notes

1. The functionality of your system will be tested automatically in the Moodle system by submitting a ZIP file with your project. You may submit as many times as you wish, though after the first six submissions, five points will be reduced from the final grade of the assignment for each additional submission.
2. To avoid wasting your trials on submissions that do not compile, there is an additional task for checking that we can compile your code. You can submit as many times as you want, with no grading on this task.
3. There will be no extension **at all!** Do not wait until the last minute to submit.
4. The Moodle deliverables must match your repository state at the time of submission. Please update it and re-push it to the repository before the final submission.

